Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

GENERATIVE AI AND AI PLATFORM ENGINEERING (25CS2207E)

2026–27, Even Semester

Section / Team Details

Section No.: 8

Team No.: 16

Project Title: Polite Feedback Mentoring Assistant using Prompt Templates and LLM-as-a-Judge Scoring

Team Members

Roll Number	Student Name	Signature
2520030395	Monish Putti	
2520030195	ADA. Rohan	

1. Abstract

This project proposes a Polite Feedback Mentoring Assistant, a Generative AI application that gives learners respectful, constructive and actionable feedback on their draft work. Feedback is generated by a large language model (LLM) using versioned prompt templates (system and user prompts) and returned as a structured output validated with a Pydantic schema. A second LLM acts as an LLM-as-a-Judge, scoring each feedback against a fixed rubric of politeness, specificity, actionability and accuracy, and giving a short reason for each score. If the score falls below a threshold, the feedback is regenerated using the judge's reason, with a limited number of retries. The judge will be calibrated against a small set of human ratings, and different prompt versions will be compared on a fixed test set using the judge's scores. Prompt version, scores, tokens, latency and cost for every call will be logged to MongoDB. The project is application-level: it combines the course's prompt engineering, structured output and evaluation concepts in one measurable pipeline. At this stage the topic selection, a literature survey of two papers and the architecture design are complete; the implementation and all results are still to be done.

2. Problem Statement

Mentors have limited time, so detailed feedback does not scale to large classes. Harsh or vague feedback can demotivate learners, and feedback drafted by an LLM may be generic, over-flattering or impolite, with tone and usefulness varying from one response to the next. There is currently no lightweight, repeatable way to check the tone and usefulness of such feedback, and prompt changes are often made by intuition rather than measurement. Existing work on LLM judges, such as Zheng et al. (2023) and Liu et al. (2023), studies general chat answers and summarisation and reports biases such as position, verbosity and self-enhancement bias, but does not address the politeness

and actionability of mentoring feedback. This is the gap identified from our current two-paper survey, which is still ongoing. The project aims to develop an assistant that generates polite, structured feedback from versioned prompt templates and evaluates it with a rubric-based, human-calibrated LLM-as-a-Judge.

3. Objectives

1. To design versioned prompt templates for polite, constructive feedback.
2. To define a rubric covering politeness, specificity, actionability and accuracy.
3. To build an LLM judge that scores each feedback and gives a reason.
4. To calibrate the judge against a small set of human ratings and report the agreement.
5. To compare prompt versions using judge scores on a fixed test set.
6. To expose the assistant through a simple demo interface.

4. Proposed Methodology

The project will be implemented in Python in the following stages, all of which are proposed work. (1) A mentee submits a draft along with context and the desired level. (2) A prompt registry stores versioned system and user templates (v1, v2, and so on), which build the request to the generator LLM. (3) The generator returns structured feedback (strengths, areas for improvement and suggested actions), which is validated with a Pydantic schema. (4) The LLM judge scores the feedback on the rubric using form-filled, rubric-style prompts and returns scores with a reason. (5) If the score is above the threshold, the feedback is shown; otherwise it is regenerated using the judge's reason, with limited retries. Each call is logged to MongoDB with the prompt version, scores, tokens, latency and cost. A small set of human ratings of sample drafts will be used to check agreement between the judge and people, and a fixed test set of self-written sample drafts will be used to compare prompt versions. The judge's known weaknesses, such as position, verbosity and self-enhancement bias, will be handled by fixing the rubric, calibrating against human ratings and reporting the limitations. The drafts will contain no sensitive personal data, and a human mentor remains in the loop.

5. Generative AI Concepts / Techniques / APIs Used

Concept / Technique / API	**Purpose in the Project**
System and user prompts	Define the mentor role and polite tone, and carry the mentee's draft.
Prompt templates and versioning	Keep prompts as versioned files so versions can be compared.
Structured output (Pydantic)	Enforce a fixed feedback format and validate model responses.
Validation, retry and safe fallback	Handle invalid or low-scoring outputs through limited retries.
LLM-as-a-Judge	Score feedback on the rubric and give a reason for each score.
Evaluation rubric	Politeness, specificity, actionability and accuracy as scoring criteria.
Human calibration	Compare a small set of human ratings with judge scores.

LLM API / local model (Ollama)	Generate feedback and judge scores; log tokens, latency and cost.
MongoDB logging	Store prompt version, scores, tokens, latency and cost per call.

6. Individual Contribution

Roll Number	**Student Name**	**Individual Responsibility**
2520030395	Monish Putti	1) Prompt registry, versioned templates and the feedback generator with Pydantic structured output. 2) Integration of project modules and documentation.
2520030195	A. Rohan	1) Rubric design, LLM judge and the regenerate-on-low-score loop. 2) Human calibration set, test set, MongoDB logging and result verification.

7. Tools / Platforms / Software Used

Tool / Platform / Software	**Purpose**
Python	Used to implement the prompt pipeline, generator, judge and evaluation scripts.
Pydantic	Used to define and validate structured feedback and judge-score outputs.
Hosted LLM API / Ollama (local)	Used to run the generator and the judge.
FastAPI	Used to expose the assistant as a service.
MongoDB	Used to log prompt version, scores, tokens, latency and cost.
React + Vite (optional)	Used for a simple demo interface if time permits.
GitHub and GitHub Actions	Used for version control and automated checks.

8. Expected Outcome

The project is expected to produce a working prototype that takes a mentee's draft and returns polite, structured and actionable feedback, scored by a rubric-based LLM judge that has been calibrated against a small set of human ratings. It will include versioned prompt templates, a fixed test set, per-call logs and a comparison of prompt versions using judge scores. The team will report how closely the judge agrees with human ratings, along with limitations such as judge bias and the small number of human raters. The novelty is application-level: a focused, measurable pipeline for mentoring feedback, not a new judging algorithm or benchmark. No implementation results exist yet, and all results will be reported as actually measured.